Modelo Relazionale

Il Modello Relazionale

- Introdotto da **Codd nel 1970** con l'articolo *A Relational Model of Data for Large Shared Data Banks*.
- L'obiettivo era **favorire l'indipendenza dei dati**, cioè separare:
  - la **rappresentazione logica** (relazioni, attributi, tuple)
  - dall'**implementazione fisica** (come i dati vengono memorizzati su disco).
- Si basa sul concetto matematico di **relazione** (dalla teoria degli insiemi).

Nella pratica, ogni **relazione** ha una **rappresentazione naturale tramite una tabella**:

- **intestazioni di colonna** = attributi (campi)
- **righe** = tuple (istanze di dati)
- **celle** = valori presi da domini.

Relazione Matematica

Una **relazione matematica** è definita così:

- Dati **n insiemi** $D_1, D_2, \ldots, D_n$ (chiamati **domini**), il **prodotto cartesiano** è:

  $$D1 \times D2 \times \cdots \times Dn = \\ (d1, d2, \ldots, dn) \mid d1 \in D1, d2 \in D2, \ldots, dn \in Dn$$

- Una **relazione su** $D_1, \ldots, D_n$ è un **sottoinsieme** del prodotto cartesiano:

  $$r \subseteq D_1 \times D_2 \times \cdots \times D_n$$

In altre parole: una relazione è un **insieme di n-uple** (tuple), dove ciascun elemento proviene da un dominio specifico.

Esempio

Supponiamo di avere due domini:

- $D_1 = \{a, b\}$
- $D_2 = \{x, y, z\}$

Prodotto cartesiano:

$D_1 \times D_2 = \{(a, x), (a, y), (a, z), (b, x), (b, y), (b, z)\}$

Relazione r = un sottoinsieme del prodotto cartesiano, ad esempio:

$r = (a, x), (a, z), (b, y)$

Dal modello matematico al modello relazionale

- In un **DB relazionale**, i domini corrispondono ai **tipi di dati** (es. interi, stringhe, date).
- Una **relazione matematica** corrisponde a una **tabella**.
- Le **n-uple** corrispondono alle **righe (tuple)** della tabella.
- I **domini** diventano gli **attributi (colonne)** della tabella.

👉 Quindi, il passaggio è:

$$Relazione\ matem. \longrightarrow Tabella\ DB$$

Definizioni di base

- **Tipo ennupla T**: insieme finito di coppie (Attributo, Tipo elementare)

  Esempio:

  $$T = (Matricola : int, Nome : string, Corso : string)$$

- **Schema di relazione R(T)**: coppia costituita dal nome della relazione R e dal tipo ennupla

  Esempio:

  $$Studenti(Mat : int, Nome : string, Corso : string)$$

- **Schema di base di dati**: insieme di schemi di relazione

  $$R1(T1), R2(T2), \ldots, Rk(Tk)$$

- **Istanza di schema R(T)**: insieme finito di ennuple di tipo

  Esempio:

  $$r = \{(Mat := 1, Nome := "Anna", Corso := "DB")$$
  $$(Mat := 2, Nome := "Luca", Corso := "AI")\}$$

- **Cardinalità di una relazione**: numero delle ennuple contenute.

  Nell'esempio sopra: $cardinalità = 2$.

- **Istanza di schema relazionale**: collezione di istanze, una per ciascuno schema di relazione.

---

2. Proprietà

- L'**ordine degli attributi non è significativo** → $(Matricola, Nome)$ è lo stesso di $(Nome, Matricola)$, se hanno stessi nomi e tipi.
- Due **tipi ennupla** sono uguali se coincidono in grado, attributi e tipi degli attributi.
- Due **tipi relazione** sono uguali se hanno lo stesso tipo ennupla.

---

3. Esempio completo

Supponiamo uno schema relazionale di un'università:

- Relazione **Studenti**:

  $Studenti(Mat : int, Nome : string, Corso : string)$

- Relazione **Esami**:

  $Esami(Mat : int, Corso : string, Voto : int)$

Istanza di Studenti:

Mat	Nome	Corso
1	Anna	DB
2	Luca	AI
3	Marco	DB

Studenti

Nome	Matricola	Provincia	AnnoNascita
Isaia	071523	PI	1982
Rossi	067459	LU	1984
Bianchi	079856	LI	1983
Bonini	075649	PI	1984

Esami

Materia	Candidato*	Data	Voto
BD	071523	12/01/06	28
BD	067459	15/09/06	30
FP	079856	25/10/06	30
BD	075649	27/06/06	25
LMM	071523	10/10/06	18

Tabelle e Relazioni

Condizioni perché una tabella rappresenti una relazione

Una tabella (cioè una relazione in senso pratico) deve rispettare alcune proprietà fondamentali:

1. **Omogeneità dei valori in colonna** → ogni colonna ha un *tipo* ben definito (es. interi, stringhe, date).

   Esempio: nella colonna *Matricola* ci devono essere solo interi.

2. **Unicità delle righe** → nessuna riga può essere duplicata.
3. **Unicità delle intestazioni di colonna** → ogni attributo ha un nome distinto.
4. **Irrilevanza dell'ordine delle righe** → cambiare l'ordine dei record non cambia la relazione.
5. **Irrilevanza dell'ordine delle colonne** → la relazione è definita dagli attributi, non dalla loro posizione.

Schema vs Istanza

In un database relazionale distinguiamo sempre due livelli:

- **Schema relazionale (intensionale)**
  - È la *struttura logica* del database, relativamente stabile nel tempo.
  - Descrive le tabelle, i nomi degli attributi e i tipi di dato.
  - È come il *progetto architettonico* di una casa.
  - Esempio:

    ```
    Studenti(Mat: int, Nome: string, Corso: string)
    ```

- **Istanza (estensionale)**
  - Sono i dati *effettivamente contenuti* nelle tabelle in un certo momento.
  - Cambiano dinamicamente nel tempo (inserimenti, aggiornamenti, cancellazioni).
  - È come l'*arredamento della casa*: può cambiare senza alterare il progetto.
  - Esempio:

Mat	Nome	Corso
1	Anna	DB
2	Luca	AI
3	Marco	DB

Il Modello Relazionale è Basato su Valori

Il modello relazionale non utilizza puntatori o riferimenti fisici diretti tra dati, ma si basa **solo sui valori** dei domini che compaiono nelle ennuple (tuple).

- I legami tra relazioni diverse sono stabiliti tramite attributi che hanno valori in comune (tipicamente le **chiavi esterne**).
- Questo garantisce che la rappresentazione sia completamente logica, indipendente dal livello fisico.

Vantaggi della Struttura Basata su Valori

1. **Indipendenza dalle strutture fisiche**
   - Le relazioni non dipendono dall'organizzazione fisica dei dati (es. file, indici, blocchi).
   - Anche se il DBMS cambia internamente il modo in cui memorizza i dati, la rappresentazione logica rimane invariata.
2. **Separazione tra logico e fisico**
   - La rappresentazione logica è costituita unicamente dai valori, senza riferimenti a strutture interne.
3. **Rilevanza applicativa**
   - Si rappresenta solo ciò che interessa all'applicazione (gli attributi e i valori).
4. **Portabilità**
   - I dati possono essere spostati facilmente tra sistemi diversi, perché non dipendono da puntatori o strutture interne al DBMS.

5. **Assenza di direzionalità vincolata**
    - Con i puntatori tradizionali il collegamento è direzionale. Con i valori, invece, le relazioni sono “bidirezionali”: se due relazioni condividono un attributo, posso navigare i dati in entrambe le direzioni.

Informazione Incompleta

Il modello relazionale richiede che i dati rispettino la struttura definita dagli schemi di relazione. Tuttavia, nella pratica, capita di dover rappresentare **informazioni incomplete**.

Principi

- Tutte le informazioni devono essere rappresentate tramite **ennupla**.
- Le ennuple devono avere un formato coerente con lo schema di relazione.

Il Valore Nullo (NULL)

- Rappresenta l’assenza di un valore di dominio.
- Non appartiene al dominio stesso: è un “valore speciale” che indica mancanza di informazione o non applicabilità.
- Formalmente: per ogni attributo `A`, il valore `t[A]` (cioè il valore di `A` in una tupla `t`) appartiene a `dom(A)` oppure può essere `NULL`.

Tabella **Persona**

IdPersona	Stato civile	Coniuge
936463	celibe	NULL
936462	sposato	936465
936465	sposata	936462

- `IdPersona` è la chiave primaria.
- Il campo `Coniuge` fa riferimento a un altro `IdPersona` nella stessa tabella (auto-relazione).

- `NULL` indica assenza di valore (es. nessun coniuge per i celibi).

Matricola	Esame	Voto	Idoneità
936463	Inglese	NULL	Si
936462	Basi di dati	30	NULL
936463	Programmazione	28	NULL

- `Matricola` identifica lo studente (chiave esterna verso `Persona`).
- `Voto` contiene un valore numerico quando l'esame è valutato.
- `Idoneità` contiene "Si" quando l'esame è superato senza voto numerico.
- `NULL` serve a rappresentare valori non applicabili o mancanti (es. idoneità non rilevante se c'è un voto).

Vincoli di Integrità

Definizione

- Uno **schema relazionale** = insieme di schemi di relazione + insieme di vincoli d'integrità.
- Un **vincolo d'integrità** = proprietà che deve essere soddisfatta da tutte le istanze valide della base di dati.
- Formalmente: un **predicato booleano** → restituisce `vero` se l'istanza è valida, `falso` se non lo è.

Perché servono

1. Rappresentano meglio la realtà (descrizione accurata).
2. Migliorano la **qualità dei dati** (evitano valori senza senso).
3. Aiutano nella **progettazione** del database.
4. Vengono usati dal **DBMS** per ottimizzare interrogazioni.

5. Non tutte le proprietà reali sono però esprimibili come vincoli formali.

Classificazione dei Vincoli

Tipo	Definizione	Esempio
Intrarelazionali	Vincoli all'interno di una singola tabella	- Vincoli di **dominio**: `DataNascita` $\in$ (valori validi di date)- Vincoli di **ennupla**: `Età ≥ 18`
Interrelazionali	Vincoli che riguardano valori presenti in più tabelle	- Vincolo di integrità referenziale: ogni `Matricola` in **Esami** deve esistere in **Persona**

Vincoli di Ennupla

- Si applicano **a ciascuna riga** della tabella (indipendenti dalle altre).
- Categorie:
  - **Vincoli di dominio**: valori ammessi per un singolo attributo.
    - Esempio: `Sesso ∈ {'M', 'F'}`
  - **Vincoli generali di ennupla**: condizioni su più attributi della stessa riga.
    - Esempio: `DataAssunzione ≥ DataNascita`

Identificazione Univoca delle Ennuple

- Serve a distinguere in modo inequivocabile ogni riga (ennupla) della tabella.
- Esempi:
  - `Matricola` (nessun duplicato ammesso → ogni studente ha la sua matricola).
  - `(Cognome, Nome, DataNascita)` (la combinazione non può ripetersi).

Vincoli di Chiave

Concetto	Definizione	Esempio
Superchiave	Insieme di attributi che identificano **univocamente** ogni ennupla.	`{Matricola}{Cognome, Nome, DataNascita}{Matricola, Nome}`
Chiave	Superchiave **minimale**: se togli un attributo, non identifica più univocamente.	`{Matricola}{Cognome, Nome, DataNascita}`
Chiave primaria	Una chiave scelta tra le possibili chiavi candidate, solitamente la più semplice.	`Matricola` (preferita a `(Cognome, Nome, DataNascita)` perché è più corta).
Attributo primo	Ogni attributo che fa parte di una chiave.	`Matricola`, oppure `Cognome`, `Nome`, `DataNascita` se usati in combinazione.

Schema Esempio

Studente(Matricola, Cognome, Nome, DataNascita, CorsoDiLaurea)

- Superchiavi possibili:
  - `{Matricola}`
  - `{Cognome, Nome, DataNascita}`
  - `{Matricola, Cognome}`
  - `{Matricola, Nome}`
- Chiavi candidate:
  - `{Matricola}`
  - `{Cognome, Nome, DataNascita}`
- Chiave primaria scelta:
  - `{Matricola}`

Tabella Studente

Matricola	Nome	Cognome	Corso	Nascita
27655	Mario	Rossi	Ing Inf	5/12/78
78763	Piero	Neri	Ing Mecc	10/7/79
65432	Mario	Neri	Ing Inf	3/11/76
87654	Mario	Rossi	Ing Inf	3/11/76
67653	Piero	Rossi	Ing Mecc	5/12/78

Tabella 1 – Esempio con valori nulli (chiave non valida)

Matricola	Nome	Cognome	Corso	Nascita
78763	Piero	Neri	Ing Mecc	NULL
65432	Mario	Neri	Ing Inf	5/12/78
87654	NULL	Rossi	Ing Civile	10/7/79
NULL	Mario	Neri	NULL	3/11/76
43289	Mario	NULL	Ing Inf	5/12/78

Analisi:

- La presenza di valori nulli in `Matricola` impedisce di identificarle univocamente.
- Non è possibile usare `Matricola` come chiave primaria.

Tabella 2 – Chiave primaria valida

Matricola	Nome	Cognome	Corso	Nascita
78763	Piero	Neri	Ing Mecc	5/12/78
65432	Mario	Neri	Ing Inf	10/7/79
87654	Mario	Rossi	Ing Civile	3/11/76
43289	Mario	Neri	Ing Inf	5/12/78
86765	Mario	Mario	Ing Inf	NULL

Analisi:

- `Matricola` è ora una **chiave primaria**: valori univoci e **senza null**.
- Permette l'identificazione univoca di ogni ennupla e i riferimenti tra relazioni.

Concetto di Integrità Referenziale

- Le tabelle (relazioni) possono essere correlate tra loro tramite valori comuni, di solito le **chiavi primarie**.
- Una tabella che fa riferimento a un'altra tramite la chiave primaria usa una **chiave esterna** (foreign key).
- La **coerenza dei dati** deve essere garantita: ogni valore della chiave esterna deve esistere nella chiave primaria della tabella di riferimento.

Esempio

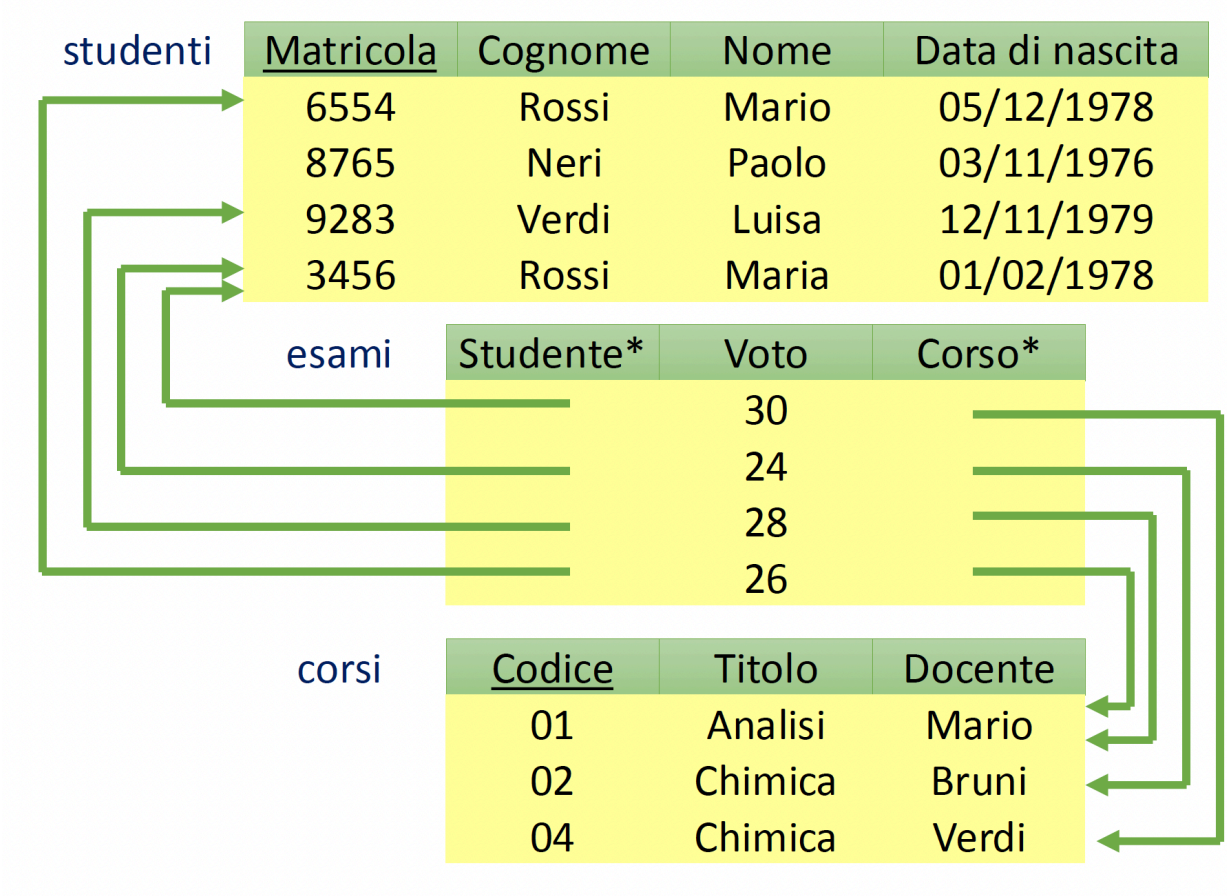
Studenti

Nome	Matricola	Provincia	AnnoNascita
Isaia	071523	PI	1982
Rossi	067459	LU	1984
Bianchi	079856	LI	1983
Bonini	075649	PI	1984



Esami

Materia	Candidato*	Data	Voto
BD	071523	12/01/06	28
BD	067459	15/09/06	30
FP	079856	25/10/06	30
BD	075649	27/06/06	25
LMM	071523	10/10/06	18



Vincolo di Integrità Referenziale

- È un vincolo che collega due relazioni (tabelle) del database.
- Definisce che **i valori di un insieme di attributi X in una relazione R1** devono comparire **come valori della chiave primaria di un'altra relazione R2**.
- In pratica, R1 "fa riferimento" a R2 tramite la chiave primaria di quest'ultima.

Infrazioni

Codice	Data	Vigile*	Stato	Numero
34321	1/2/17	3987	Fra	CB 123 AA
53524	4/3/18	3295	Ita	AB 222 CF
64521	5/4/17	3295	Ita	AX 424 DA
73321	5/2/19	9345	Fra	AX 424 DA

Vigili

Matricola	Cognome	Nome
3987	Rossi	Luca
3295	Neri	Piero
9345	Neri	Mario
7543	Mori	Gino

Azioni compensative in caso di violazione del vincolo

Quando un'operazione (es. cancellazione di una riga in AUTO o VIGILI) viola un vincolo referenziale, i DBMS possono intervenire con:

1. **Rifiuto dell'operazione**: l'eliminazione non viene eseguita.
2. **Eliminazione in cascata**: tutte le righe collegate nelle tabelle dipendenti vengono eliminate automaticamente.
3. **Introduzione di valori nulli**: gli attributi foreign key diventano NULL, se consentito.

Eliminazione in Cascata

Impiegati

Matricola	Cognome	Progetto*
34321	Rossi	IDEA
53524	Neri	XYZ
64521	Verdi	NULL
73032	Bianchi	IDEA

Progetti

Codice	Inizio	Durata	Costo
IDEA	01/2000	36	200
XYZ	07/2001	24	120
BOH	09/2001	24	150

Introduzione di Valori Nulli

Impiegati

Matricola	Cognome	Progetto*
34321	Rossi	IDEA
53524	Neri	NULL
64521	Verdi	NULL
73032	Bianchi	IDEA

Progetti

Codice	Inizio	Durata	Costo
IDEA	01/2000	36	200
XYZ	07/2001	24	120
BOH	09/2001	24	150

Vincoli Multipli su più Attributi

Incidenti

Codice	Data	StatoA*	NumeroA*	StatoB*	NumeroB*
34321	1/2/95	Ita	AB 222 CF	Ita	AX 424 DA
64521	5/4/96	Ita	AX 424 DA	Fra	CB 123 AA

Auto

Stato	Numero	Cognome	Nome
Ita	AX 424 DA	Bianchi	Giovanni
Ita	AB 222 CF	Rossi	Mario
Fra	CB 123 AA	Jacques	Dupont